{ git }

&

GitHub

Controlando seu Código

Ambientes Virtuais · Versionamento · Boas Práticas

Prof: Cassio Ribeiro - SENAC RJ

O que é Git?

CONTROLE DE VERSÃO

- Sistema de controle de versão instalado na sua máquina
- Rastreia todas as alterações feitas nos arquivos ao longo do tempo
- Funciona completamente offline, sem precisar de internet
- Criado por Linus Torvalds em 2005 para o kernel Linux

Analogia

"É o histórico de versões do seu projeto, salvo localmente — como o Ctrl+Z com superpoderes."

O que é GitHub?

PLATAFORMA ONLINE

- Plataforma online que hospeda repositórios Git na nuvem
- Permite colaboração entre equipes de qualquer lugar do mundo
- Funciona como portfólio público para desenvolvedores
- Pertence à Microsoft desde 2018

Analogia

"É o Google Drive dos programadores, mas com superpoderes de colaboração e histórico."

Git vs GitHub

	Git	GitHub
Onde roda?	Na sua máquina	Na nuvem
Internet?	Não precisa	Sim, é online
O que faz?	Versiona arquivos	Hospeda repositórios
Quem criou?	Linus Torvalds (2005)	Tom Preston-Werner (2008)
Trabalha sozinho?	Sim, 100% local	Depende

Por que o nome do repositório importa?

PORTFÓLIO PROFISSIONAL

`github.com/` **`seu-usuario`** ← *seu endereço profissional!*

- ✅ Use nomes claros e em minúsculo
- ✅ Separe palavras com hífen: analise-de-dados, projeto-final
- ❌ Evite espaços, acentos e caracteres especiais
- 🌐 Recrutadores e professores acessam seu perfil público

Criando um Repositório no GitHub

1. Clique no botão verde "New" na página inicial
2. Defina o nome do repositório (ex: analise-de-dados)
3. Escolha entre Público ou Privado
4. Marque "Add a README file" ← sempre!
5. Clique em "Create repository"

🔓 Público

Qualquer pessoa pode ver. Ideal para portfólio.

🔒 Privado

Somente você (e convidados) tem acesso.

Configuração Inicial do Git

FEITO UMA VEZ POR MÁQUINA

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

user.name

Identifica quem fez cada commit no histórico

user.email

Use o mesmo e-mail cadastrado no GitHub

--global

Aplica a configuração para todos os projetos da máquina

Verificar

Use git config --list para conferir as configurações

git clone

BAIXAR UM REPOSITÓRIO

```
git clone https://github.com/usuario/repositorio.git
```

O que acontece: Cria uma pasta local com todo o projeto + histórico de commits

GitHub (nuvem) → **git clone** → **Pasta local**

💡 *Analogia: "É como dar Download no projeto inteiro, com todo o histórico de alterações."*

O Fluxo Básico

ADD → COMMIT → PUSH

`git add .`	Seleciona quais arquivos entram no próximo commit
`git commit -m ...`	Salva um ponto no histórico com uma mensagem descritiva
`git push`	Envia os commits para o GitHub (repositório remoto)

git add

PREPARAR ARQUIVOS

```
git add arquivo.py        # adiciona um arquivo específico
git add .                 # adiciona TUDO na pasta atual
git add *.csv             # adiciona todos os arquivos .csv
```

Sem git add

O arquivo foi modificado, mas ainda NÃO está selecionado para o commit

Com git add

O arquivo entra na 'área de stage', pronto para ser commitado

📦 *Analogia: "É separar o que vai na caixa antes de lacrar e enviar."*

git commit

SALVAR NA LINHA DO TEMPO

```
git commit -m "Adiciona análise exploratória dos dados"
git commit -m "Corrige cálculo de média por cidade"
```

❌ Mensagens ruins

- "update"
- "mudanças"
- "fix"
- "aaaaaa"

✅ Mensagens boas

- "Cria análise exploratória"
- "Corrige leitura do CSV"
- "Adiciona gráfico de barras"

💾 *Cada commit é um ponto na linha do tempo — você pode voltar a ele quando quiser!*

git push e git pull

SINCRONIZAR COM O GITHUB

```
git push origin main     # envia commits locais → GitHub
git pull origin main     # baixa atualizações do GitHub → local
```

🚀 git push

Sua máquina → GitHub

Envia seus commits para o repositório remoto.

⬇️ git pull

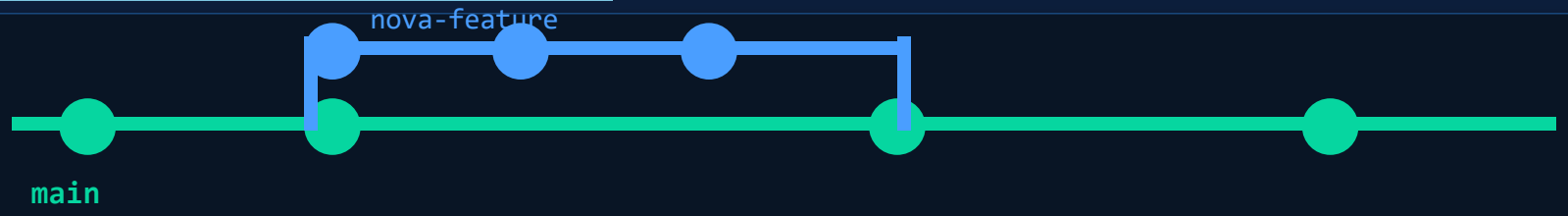
GitHub → Sua máquina

Baixa as atualizações do repositório remoto.

⚠️ Dica em equipe: sempre faça git pull ANTES do git push para evitar conflitos!

Branches: O que são?

RAMIFICAÇÕES DO PROJETO




```
git branch nova-feature          # cria a branch
git checkout nova-feature         # muda para ela
# OU em um comando só:
git checkout -b nova-feature
```

🌿 *Analogia: "É um rascunho — você experimenta à vontade e só junta ao original quando estiver pronto."*

Quando Usar Branch?

Nova funcionalidade

Desenvolvendo algo novo sem comprometer o que já funciona

Correção de bug

Conserta um erro sem arriscar o código estável em produção

Trabalho em equipe

Cada membro trabalha na sua branch sem conflitar com os outros

Experimentos

Testa novas ideias que podem ou não ser aprovadas depois

💡 *Regra prática: "Se vai mexer em algo importante, crie uma branch antes!"*

Merge: Juntando as Ramificações

INTEGRAR BRANCHES

```
git checkout main            # volta para a branch principal
git merge nova-feature       # integra as mudanças
git branch -d nova-feature   # deleta a branch após o merge
```

1. Certifique-se de estar na branch main
2. Execute git merge nome-da-branch
3. Se houver conflitos, o Git vai sinalizar onde resolver manualmente
4. Após resolver, faça um novo commit e delete a branch

O que é um Ambiente Virtual (venv)?

ISOLAMENTO DE PROJETO

🔄 Cada projeto Python tem suas próprias bibliotecas, isoladas do sistema.

❌ Sem venv

Projeto A precisa de pandas 1.5
Projeto B precisa de pandas 2.1

Conflito! Um sobrescreve o outro.

✅ Com venv

Projeto A: pandas 1.5 (na própria pasta)
Projeto B: pandas 2.1 (na própria pasta)

Sem conflito! Cada um no seu espaço.

🎒 *Analogia: "É uma mochila separada para cada projeto — cada um carrega só o que precisa."*

Criando e Ativando o venv

CONFIGURAÇÃO DO AMBIENTE

```
# 1. Criar o ambiente virtual
python -m venv venv

# 2. Ativar (Windows)
venv\Scripts\activate

# 3. Ativar (Mac/Linux)
source venv/bin/activate

# 4. Desativar
deactivate
```

- A pasta venv/ é criada dentro do projeto
- O terminal mostra "(venv)" quando ativo
- Crie o venv UMA vez por projeto, não por arquivo
- Ative sempre que abrir um novo terminal

Pacotes e requirements.txt

LISTA DE DEPENDÊNCIAS

```
# Instalar pacotes (com venv ativo!)
pip install pandas matplotlib numpy

# Ver o que está instalado
pip list

# Gerar o arquivo requirements.txt
pip freeze > requirements.txt

# Instalar a partir do arquivo (outra máquina)
pip install -r requirements.txt
```

📋 **requirements.txt**

```
pandas==2.1.0
matplotlib==3.8.0
numpy==1.26.0
scipy==1.11.0
```

Este arquivo deve ser commitado no GitHub! Ele substitui a pasta venv/ no repositório.

.gitignore: O que NÃO vai para o GitHub

IGNORAR ARQUIVOS

```
# .gitignore
venv/
__pycache__/
*.pyc
.env
.DS_Store
*.log
*.sqlite3
```

`venv/`	Pesada e desnecessária — use requirements.txt
`__pycache__/`	Gerada automaticamente pelo Python
`.env`	Contém senhas e chaves de API — NUNCA suba!
`*.pyc`	Arquivos compilados temporários

🔑 *Dica: o GitHub gera um .gitignore automático — selecione "Python" na criação do repositório!*

Fluxo Completo: venv + Git + GitHub

1

Criar projeto

```
Git Clone …
```

2

Criar venv

```
python -m venv venv
```

3

Ativar venv

```
venv\Scripts\activate
```

4

Instalar libs

```
pip install pandas
```

5

Gerar requirements

```
pip freeze >
requirements.txt
```

6

Adicionar ao Git

```
git add .
```

7

Commit

```
git commit -m
"início do projeto"
```

8

Push

```
git push
origin main
```

O que é o README.md?

APRESENTAÇÃO DO PROJETO

Página inicial do repositório

Aparece automaticamente na página do projeto no GitHub

Documentação básica

Explica: o que é, como instalar, como usar o projeto

Cartão de visitas

É o primeiro contato de recrutadores com seu projeto

Formato Markdown

.md

Arquivo de texto com marcações simples para formatação

Um bom README vale mais que mil linhas de código — é ele que convence alguém a usar seu projeto!

Markdown: Formatação Básica

SINTAXE DO MARKDOWN

```
# Título H1
## Título H2
### Título H3

**negrito**   *itálico*

- item de lista
- outro item

1. lista numerada
2. segundo item

`código inline`

[link](https://url.com)

![imagem](caminho/img.png)
```

Pré-visualização

Título H1

Título H2

negrito itálico

- item de lista
- outro item

`código inline`

🔗 link 📷 imagem

Markdown: Código e Tabelas

ELEMENTOS AVANÇADOS

```
```python
print('Olá, mundo!')
import pandas as pd
df = pd.read_csv('dados.csv')
```
```

```
| Nome | Nota |
|------|------|
| Ana  | 9.5  |
| João | 8.0  |
```

Bloco de código Python:

```python
print('Olá, mundo!')
import pandas as pd
```

Tabela Markdown:

Nome	Nota
Ana	9.5
João	8.0

Estrutura de um README Profissional

1	`# Nome do Projeto`	Título claro + breve descrição do que o projeto faz
2	`## Tecnologias`	Python, pandas, MySQL, Power BI — libs e versões usadas
3	`## Como instalar`	git clone ... → pip install -r requirements.txt
4	`## Como usar`	Exemplo de execução, prints de telas, comandos principais
5	`## Autor`	Seu nome, e-mail e link do LinkedIn/GitHub

📋 Cheatsheet — Comandos Essenciais

`git config --global user.name ...`	Define o nome do usuário
`git config --global user.email ...`	Define o e-mail do usuário
`git clone <url>`	Clona um repositório remoto
`git add .`	Adiciona todos os arquivos ao stage
`git commit -m "msg"`	Salva um ponto no histórico
`git push origin main`	Envia commits para o GitHub
`git pull origin main`	Baixa atualizações do GitHub
`git checkout -b branch`	Cria e muda para nova branch
`git merge branch`	Integra branch na main
`python -m venv venv`	Cria o ambiente virtual
`pip install -r requirements.txt`	Instala dependências do projeto
`pip freeze > requirements.txt`	Exporta dependências instaladas

Seu GitHub é seu portfólio.

Cada commit conta sua história como desenvolvedor.

Próximos passos:

- ✅ Crie sua conta no GitHub hoje
- ✅ Crie seu primeiro repositório com um README
- ✅ Faça seu primeiro commit de um projeto Python
- ✅ Compartilhe o link do seu perfil!